

Module 01 — What is dbt and Why We Use It

Tier: ● Beginner · Duration: 90 min · Prerequisites: None

Agenda

Time	Duration	Topic	Learning Goal	Mode	Participant Activity	Materials	Trainer
00:00	10 min	Welcome & overview	Understand what this module covers and what success looks like	Present	Listen, ask questions	This doc	Set expectations at the end of session, explain dbt colleague language"
00:10	20 min	The problem dbt solves	Understand why raw SQL pipelines break at scale	Present + Discussion	Ask questions, share own pain points	Whiteboard / slides	Use a concrete example: how data → BRONZE.HUB → how it is queried across systems
00:30	15 min	What dbt actually is	Understand what dbt does — and does not do	Present	Listen, annotate	Slide: dbt stack diagram	Common misconception: dbt is NOT a scheduler, It transforms data already in the warehouse
00:45	15 min	Live demo: project structure	Recognise the key files and folders in the dbt project	Demo	Watch, no coding	VS Code with dbt project open	Show real dbt project structure: dbt_project.yml, models/, macros/. Don't explain everything — just build a map. Make a deliberate mistake around naming
01:00	25 min	Exercise: explore and document	Navigate the project and find real examples	Practice	Independent exploration	dbt project (read-only)	Give workbooks. Circulate to help. Answer questions. Let them struggle a bit first
01:25	10 min	Debrief + 3-bullet summary	Consolidate learning	Debrief	Discuss, write down summary	Whiteboard	Reveal the answers only after they've tried

Theory — Part A: The Problem dbt Solves (20 min)

What happened before dbt

Here's a scenario you've probably lived. Your team wants to answer: *"How many HubSpot contacts converted to active patients last month?"*

Someone writes a SQL query. It works. They save it somewhere — a shared folder, a BI tool, their laptop. Three months later the query is broken. A column was renamed in the source. There are no tests, no documentation. Two analysts wrote the same transformation differently and got different numbers. The business has lost confidence in the data.

That's how most data teams operate without a transformation framework.

The four core problems dbt solves:

- 1. **No single source of truth** — the same calculation is defined in 10 different places
- 2. **No testing** — transformations break silently; nobody notices until a dashboard is wrong
- 3. **No documentation** — tribal knowledge about what `fct_prescription.amount` actually means
- 4. **No dependency management** — nobody knows what breaks when a source column changes

What this looks like in practice

- HubSpot pipeline stages are ingested via Lambda → `BRONZE.HUBSPOT.pipeline_stages`
- Silver models (`dim_pipeline` , `dim_pipeline_stage`) transform and clean that data
- Gold marts (`mrt_*`) serve Power BI directly
- If someone renames a column in Bronze, every downstream model that references it will break — unless dbt's dependency graph catches it

Theory — Part B: What dbt Actually Is (15 min)

The one-sentence definition

dbt is a **transformation framework** that lets you write SQL `SELECT` statements and handles materialisation, dependency resolution, testing, and documentation on top of them.

dbt does:

- Compile your SQL models into executable statements
- Resolve the order models must run in (the DAG)
- Run tests against your data
- Generate documentation

dbt does not:

- Extract data from sources (that's your Lambda / Fivetran / Airbyte)
- Load data into Snowflake (that's already done before dbt runs)
- Schedule itself (that's your orchestrator)
- Store any data itself

dbt Core vs dbt Cloud

	dbt Core	dbt Cloud
What it is	Open-source CLI tool	Hosted platform with IDE, scheduler, CI

What we use	✅ dbt Core	❌ Not used
How we run it	CLI: dbt run, dbt test, dbt build	N/A
How we schedule it	your orchestrator	N/A

Everything in this training applies to **dbt Core only**.

The dbt stack

```

flowchart TD
    SRC(["Source Systems<br/>HubSpot · Shopify · APIs"])
    LAMBDA["AWS Lambda<br/>Ingestion"]
    BRONZE["BRONZE<br/>raw · append-only<br/>Lambda owns – dbt does NOT touch"]
    STAGING["STAGING<br/>views · stg_hubspot__*"]
    SILVER["SILVER<br/>dim_* · fct_* · bridge_*"]
    GOLD["GOLD<br/>mrt_*"]
    PBI(["Power BI"])

    SRC --> LAMBDA
    LAMBDA --> BRONZE
    BRONZE -->|"source()"| STAGING
    STAGING -->|"ref()"| SILVER
    SILVER -->|"ref()"| GOLD
    GOLD --> PBI

    classDef dbt fill:#ecfdf5,stroke:#16a34a,color:#065f46
    classDef lambda fill:#fef9c3,stroke:#ca8a04,color:#713f12
    classDef bronze fill:#f1f5f9,stroke:#94a3b8,color:#475569
    class STAGING,SILVER,GOLD dbt
    class LAMBDA lambda
    class BRONZE bronze
  
```

dbt owns everything from Staging downward.

Live Demo Script (15 min)

Open VS Code with the dbt project. Walk through:

1. `dbt_project.yml` — project name, model paths, default materialisation config
2. `models/` directory — show Bronze staging, Silver, Gold folder structure
3. One Silver model — open `dim_patient.sql`, point out the `{{ ref() }}` call (don't explain it yet — just say "this is how models connect; we'll cover it in Module 03")
4. `schema.yml` — show it exists, explain it holds tests and documentation (covered later)
5. **Deliberate mistake:** navigate to the wrong folder, act confused, correct yourself — this shows how to orient yourself when lost in a real project

Do not run `dbt run`. That belongs in Module 02 after setup is confirmed.

Exercise Worksheet (25 min)

Answer all four questions using only the dbt project — no Googling.

Q1. What is the project name defined in `dbt_project.yml` ?

Q2. Find one Silver dimension model (`dim_*`). What table does it reference using `{{ ref() }}` ?

Q3. How many models are in the `models/gold/` folder? List their names.

Q4. Open `dbt_project.yml` . What is the default materialisation for Silver models?

Bonus: Run `dbt ls` in the terminal. How many models are listed?

Debrief — 3-Bullet Summary

Share your own answers first. Then confirm these three:

1. dbt transforms data that is **already in Snowflake** — it doesn't extract or load
 2. In this project, dbt owns Bronze Staging → Silver → Gold; Lambda handles ingestion
 3. Every model is a `SELECT` statement; dbt handles the `CREATE TABLE` / `CREATE VIEW` around it
-

Reference Material

- [dbt Core documentation](#)
 - [What is dbt? \(official overview\)](#)
 - Internal: `conventions` skill — layer ownership, naming rules
 - Internal: `references/go_live_checklist.md` — read before first PR
-

Prep Questions for Module 02

Answer these at the start of the next session — from memory, without notes:

1. What does `dbt_project.yml` configure?
2. What's the difference between dbt Core and dbt Cloud?
3. dbt does NOT do three things — name them.
4. Which layer does dbt own in this project?